2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC) | 979-8-3315-0233-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/VL-HCC65237.2025.00038

CODIGEN: Enhancing Personalized Programming Education with Visual Authoring and LLMs

Mohamed Ez-zaouia
University of Rennes, CNRS, IRISA - UMR 6074
Lannion, France
mohamed.ez-zaouia@univ-rennes.fr

Yazid Boumarafi
University of Rennes, CNRS, IRISA - UMR 6074
Rennes, France
yazid.boumarafi@agrocampus-ouest.fr

***Abstract*—Computer programming is an essential skill for the 21st-century workforce. However, mastering programming is challenging, especially for beginners. Recently, digital tools have become popular in supporting programming education. However, creating engaging programming materials remains difficult due to the many factors that need consideration. We introduce CODIGEN, an interactive tool powered by large language models (LLMs) that integrates visual authoring and prompt engineering to support programming education with various learning strategies. A user study with 29 participants showed that CODIGEN can assist teachers in generating quality programming materials. Statistical analysis showed that CODIGEN supports learners' confidence and learning outcomes in programming. The perceived utility and engagement with the tool were also statistically significant. We discuss our findings to guide the design and use of LLM-powered authoring tools for educational purposes.**

***Index Terms*—Authoring Tools, Programming Education, Large Language Models, Whiteboard+palettes, Intelligent User Interfaces**

I. INTRODUCTION

Computer programming is essential for the 21st-century workforce [45, 64]. Many calls are issued to foster programming education in various areas, including K-12, vocational, and higher education [7]. However, mastering programming is challenging, especially for novice learners [42, 11, 36]. Writing machine-readable instructions and syntax is hard and requires perseverance, despite difficulty, trial-and-error, and stagnations [37].

Digital tools have gained popularity. They provide small exercises that learners solve to gain feedback and progress [28, 50]. However, designing quality and personalized exercise tools is still challenging because of the combinatorial factors that need consideration when creating self-paced exercises [42, 11, 43]. This includes programming language, examples, explanations, exercises, solutions, feedback, personalization, and so forth. This challenge has led to drawbacks. One of them is the provision of a limited number of one-size-fits-all content. One-size-fits-all exercises can be limited in terms of personalization, which is shown to be effective in catering to individuals' needs [8]. Moreover, solutions are often published online. This might explain why learners still get stuck, go off task, become bored, or try to game the system when using these exercise tools [5]. Various approaches have been explored to overcome this challenge, such as creating materials through crowdsourcing [1, 52]; however, it remains an open challenge.

Recently, LLMs or GAI (Generative Artificial Intelligence) have demonstrated numerous capabilities with great potential to reshape education [12, 35]. Moreover, the capabilities of LLMs have been shown to transfer to various tasks, including writing, design, creativity, and programming [15, 18, 3]. However, interacting with LLMs is mainly done through prompting using linear conversational interfaces or chatbots. This type of interaction might not be effective for programming education for several reasons. Prompting, also known as prompt engineering, can be challenging and might require advanced techniques, such as one-shot, and few-shot methods to achieve the best results from LLMs [38, 65]. Furthermore, in-situ assistance is another limitation of the current linear interfaces of chatbots. Current LLM interfaces force the user to scroll through and make sense of (sometimes) huge chunks of text. They require the user to organize the content themselves, copying and editing it in other tools and workflows. This makes it harder for users to author, learn from and make-sense of information using existing LLM interfaces. Thus, research into in-situ assistance tools powered LLMs, beyond conversational interfaces, is timely and important, in particular, *how can we design a visual authoring tool powered by in-situ GAI assistance to support programming education?* (**RQ1**), *How do users engage with and perceive the utility of visual authoring tools powered by GAI assistance?* (**RQ2**), *how does such visual authoring tools powered by GAI impact the educational outcomes of users?* (**RQ3**), and *how do teachers and learners perceive such GAI powered tools in supporting programming education? (**RQ4**).*

To address these research questions, we first introduce a conceptual framework to guide the design of visuospatial authoring tools with in-situ GAI assistance. We then present CODIGEN, a visual authoring tool powered by LLMs to support programming education. CODIGEN offers tools to process natural language and support programming education via different learning strategies, including programming examples, explanations, and variations of fill-in-blank exercises, solutions, feedback, and personalization (we referred to this as programming materials). In contrast to other approaches that leverage LLMs to generate code using natural language [e.g., 44] or explain existing code [e.g., 46, 34], we leverage LLMs to alleviate challenges that arise from generating variations of small original explanations, examples, exercises, and feedback that cater to individual needs. In addition to teachers, CODIGEN enables learners to author educational materials, which is highly ranked in Bloom taxonomy [30].

1943-6106/25/$31.00 ©2025 IEEE
DOI 10.1109/VL-HCC65237.2025.00038

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

We conducted a user study of CODIGEN with 29 participants in higher education (HE). The results showed a positive response from participants. The findings suggest that CODIGEN can effectively assist teachers in generating high-quality programming materials. Statistical analysis showed that CODIGEN supports learners' confidence and learning outcomes in programming, in a self-paced and personalized manner. The perceived utility and engagement with the tool were statistically significant. While we suspect that CODIGEN can be used at various levels (e.g., middle school, high school, vocational education), we evaluated it only in HE. We discuss our findings to guide the design and use of LLM-powered authoring tools for education. We extend prior research through the following contributions:

- A conceptual framework to support quality learning through visuospatial authoring tools powered by in-situ GAI assistance.
- CODIGEN, a flexible authoring tool powered by GAI, enabling the creation of programming and other educational materials.
- A user study demonstrating that integrating LLMs into authoring tools can provide effective in-situ support for teachers and learners in programming education.

II. BACKGROUND AND RELATED WORK

We build upon prior research, in particular, materials and tools for programming education, LLMs and prompt engineering, and multimodal interactions with LLMs.

A. Practice Exercises for Computer Programming Education

Programming or writing instructions in the form of machine-readable syntax is challenging [42, 11]. To acquire mastery in programming, learners need to engage in activities that foster active practice and reflection [4, 56], which requires creation of various programming materials, such as explanations, examples, and exercises [51]. Moreover, research suggests that learning activities are most effective when personalized, depending on contextual factors such as learners' needs, interests, and levels [8]. Further, learning is more effective when immediate feedback is provided to learners [66]. Thus, learners need methods and tools to support meta-cognitive skills, such as self-motivation, self-regulation, self-reflection, and perseverance [51, 56].

However, creating and administering a large variety of engaging programming activities that cater to learners' individual differences is time-consuming and difficult [42, 11]. To alleviate this challenge, small exercise tools have become increasingly popular [50]. They provide a variety of small exercises that learners practice, develop mastery at their own pace, and receive immediate feedback [28], which can help them reflect on their progress. However, existing small exercise tools have limitations, mainly related to combinatorial factors that need consideration, including the learners' levels, the degree of personalization, the programming concepts, examples, explanations, exercises, solutions, types of feedback, and so forth [11, 13]. Moreover, exercises are mainly designed by experts and scripted [43, 50], offering one-size-fits-all materials that can lack personalization. Further, solutions are often published online. This might explain why learners still get stuck, go off task, become bored, or try to game the system when using these exercise tools [5]. Various approaches have been explored to overcome this challenge, such as creating materials using crowdsourcing [1, 52], but designing original and personalized exercises remains an open challenge.

We harness GAI to generate original variations of programming materials through visual interactions. Our approach streamline and simplifies the creation of materials, including explanations, examples, exercises, and solutions. It also tailors the materials to learners' levels: beginner, intermediate, and advanced. And, it personalizes the materials to various domains of interest: gaming, music, art and graphics, social media, sports and fitness, cooking and recipes, environmental sustainability, and travel and exploration [74]. The generated materials can be directly edited and refined to best suit the needs and contexts of teachers and learners. Finally, we use GAI to provide learners with on-the-spot and immediate feedback and guidance [66].

B. LLMs and Prompt Engineering

GAI offers many capabilities for education [12, 35] in various tasks, including writing, design, creativity, and programming [15, 18, 3]. LLMs can analyze and generate text, images, audio, and video based on simple prompts as well as domain-specific content, e.g., programming code, LaTeX, JSON [32, 47].

However, interactions with LLMs are primarily designed as conversational-based interactions [e.g., 48]. Such linear interfaces, make it harder for users to visualize, filter, analyze, extend, and develop a deeper understanding of interrelated ideas and concepts. Moreover, conversational interfaces force the user to scroll and read through sometimes huge chunks of text to make sense of the outputs. Further, they require the user to organize the output content themselves, copying and editing it for use in other contexts. There is a pressing need for tools that support in-situ assistance by integrating GAI within the files, workflows, and digital workspaces of users. Another drawback is that interacting with LLMs using text-based prompts can be challenging [38, 65]. This technique requires the user to self-select and devise the right wordings that might trigger an LLM to generate responses best. However, users mostly use a simpler, often not effective, form of prompting: zero-shot.Other approaches, such as one-shot or few-shot, involve providing examples of the relevant task and context, which can be challenging.

In this paper, instead of using LLMs to generate code [e.g., 44] or explain existing code [e.g., 46, 34], we aim to integrate LLMs within unstructured and non-linear user interfaces to support programming education. We integrate prompting techniques to enhance user interaction with LLMs for programming education. We aim to assist users in directing the content generation of LLMs effectively by combining smart-default parameters, interactive menus, and few-show prompting, allowing users to easily customize and generate materials tailored for programming education. Moreover, we incorporate LLMs into text editors, coding editors, and a digital whiteboard. We aim to minimize the cognitive distance between the user's context and AI assistance. This integration is crucial because LLMs are known to produce inaccurate or misleading information [6]. Our approach allows users to extend and integrate LLMs generated content to fit their needs and contexts.

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

C. Tools for Computer Programming Education

Amoozadeh et al. [2] studied how university students used ChatGPT to complete Python programming tasks in the VSCode IDE with the GPT plugin. They found that around 33% of students attempted to solve the task by submitting the full task description to GPT without making any effort on their own. Moreover, few students took the time to verify their solutions. Another study [26] of GAI for code found that more than half of all requested prompts were copied directly from the task description and that poorly crafted prompts led to low-quality code that learners struggled to work with. This suggests that guardrails are needed for GAI-powered educational tools.

Ivie [69] leverages programming assistants to help developers inspect AI-generated code. GILT [46] integrates a similar approach by placing assistants directly in VSCode, finding that GILT supports students better compared to Google search. CodeHelp [34] and CodeAid [27] use LLMs where users can submit prompts, code snippets, and code errors to get assistance without directly revealing solutions.

While these tools support programming, they are mainly designed as structured IDEs and do not combine generation of personalized code explanations, examples, exercises, and feedback into a single unstructured interface. Our approach also differs from existing commercial tools, such as GitHub Copilot or Cursor [e.g., 44], because they are designed to generate and complete code. Our approach provides authoring tools [e.g., 73] powered by GAI to support teachers and learners author programming materials using visuospatial interactions.

D. Multimodal Interactions With LLMs

As LLMs become more popular, research into new multimodal interaction techniques beyond linear interaction is becoming increasingly important to support in-situ assistants in various contexts. Two types of user interfaces coupled with LLMs have been proposed. The first set of tools is designed for experts, like developers, to create and work with LLMs. These tools make it easier to train and test LLMs without needing complex coding [e.g., 23]. The second type of tools is designed for general public users.Wordcraft [70] uses a text editor to let writers work together with an LLM, showing that it helps create new writing experiences. Idea Machine [16] lets users interact with ChatGPT for suggestions and idea combinations, and also lets them track and revert their ideas if necessary.

Whiteboards coupled with LLMs are also proposed to help people understand and work with information better. Graphologue [24] changes long LLM responses into easy-to-read diagrams, which can be helpful for finding information. Jamplate [68] provides a digital whiteboard with structured prompts to help beginners think and organize their ideas. Sensecape [57] offers a way to manage information in a more flexible, visual way instead of just chatting linearly.

While the aforementioned tools and studies highlight the benefits of visuospatial interactions for sense-making tasks, they were not specifically designed for programming education. Furthermore, the tools discussed are primarily focused on either expert developers of LLMs or a general audience, with little emphasis on catering specifically to learners or teachers in computer programming. The unique needs of learners, such as the need for feedback [e.g., 66], process structuring [e.g., 72], and visual representation of interrelated ideas and concepts [e.g., 61], have yet to be examined using the capabilities of LLMs.

III. Conceptual Framework

To inform the design and use of visual authoring tools powered by in-situ GAI assistance for programming education (**RQ1**), we articulate a conceptual framework that outlines the theoretical rationales and the design principles that support them.

A. Theoretical Rationales

Digital content offers valuable features for learning—such as multimodality, interactivity, and dynamicity [21]—across formats like text, images, PDFs, audio, video, animations, widgets, visualizations, and annotations. However, key challenges remain: (i) supporting the authoring of personalized, multimodal materials (e.g., explanations, examples, exercises, multilingual feedback); (ii) helping users grasp content structures and concept relationships [25]; (iii) guiding attention to important information [59]; and (iv) promoting deeper, active processing of information [63]. Designing digital content to foster deep rather than superficial learning is therefore essential.

To address these challenges, we draw on factors identified in prior research that foster high-quality learning. Three key rationales (R[1–3]) guide our approach: generative multimodal learning, visual design, and active information processing.

R1) Multimodal generative learning: Quality learning theory stresses integrating various documents and resources to construct a coherent understanding [29]. Each document provides a single viewpoint. Thus, relying on a single source is insufficient for comprehensive understanding. Generative learning strategies (GLS) are crucial for supporting quality learning [17]. GLS include concept mapping, explaining, summarizing, testing, questioning, and drawing. GLS enhance learning by promoting active construction and development of relationships from learned information. Moreover, multimodal representations also support quality learning by engaging different information processing pathways. The dual-coding theory, or encoding information through multiple modalities, such as text and visuals, creates different cognitive channels for better information consolidation and retrieval [41]. Further, this approach also enhances attention to content and engagement [20].

R2) Visual design: Gestalt theory provides important principles for recognizing meaningful structures and patterns in visual scenes, based on scene segmentation and grouping objects based on similarities, proximity, contiguity, and common fate [60]. Building on these principles, the signalizing principle enhances processing by using visual attributes like color and size to emphasize key information [59], thereby reducing search time and improving information interpretation. Applying these principles may include breaking complex content into smaller parts, organizing information spatially, grouping similar elements together, highlighting hierarchical relationships, and emphasizing core information. These approaches can facilitate comprehension

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

[25]. Moreover, aligning with cognitive load theory, such design ensures that learning materials do not overwhelm learners' working memory [58].

R3) Active information processing: Research has established a distinction between passive and active forms of learning, with the latter promoting deeper learning [63]. Active learning is a self-directed approach where learners exert control over their learning by engaging with materials to select, sort, analyze, and extend their understanding [19]. Evidence suggests that this active involvement enhances information processing and leads to improved comprehension and performance [e.g., 55, 54]. For instance, individuals who created visual representations before making decisions demonstrated greater accuracy, indicating that such representations facilitate more effective information processing. Digital environments can engage learners with materials, e.g., to annotate and highlight information, compare, contrast, and organize ideas and concepts [19].

B. Design Principles

We identified three principles to support the rationales R[1-3].

DP1) Promote unstructured interaction: Quality learning requires a combination of personalized activities rather than a one-size-fits-all approach [29]. Learning is contextual and can involve diverse tasks, such as brainstorming, annotating, summarizing, and testing (**R1**). While structured tools like typed documents (e.g., IDEs) are commonly used, they often impose limitations on user interaction. Instead, unstructured interactions can foster active learning and reflection (**R3**). They enable users to create and extend information, serving as notes and annotations for collaboration and feedback. Unstructured authoring tools, such as rich text editors, marking tools, and digital whiteboards can help users filter information, track progress, organize information, and personalize their learning experiences [14].

DP2) Promote in-situ GAI capabilities: To optimize GAI for education, authoring tools can integrate LLMs and prompt engineering within unstructured interfaces. GAI can enable multimodal content creation [41], such as explanations, exercises, and illustrations (**R1**). It can facilitates collaborative tasks like feedback, brainstorming, writing, and problem solving, encouraging learners to interact with concepts in engaging manner [27] (**R3**). By embedding LLMs into text editors and whiteboards, we can bridge the gap between user context and AI support. LLMs can produce inaccuracies [6], thus allowing users to modify and adapt AI-generated content to meet their needs and contexts is crucial.

DP3) Support visuospatial representations of materials: Concept mapping, a method for visualizing interrelated ideas [71], has emerged as an effective technique for knowledge inquiry for education, creativity, and sense-making (**R2**). These node-link representations illustrate concepts (nodes) and their relationships (links), serving as a key strategy for generative multimodal learning (**R1**). One study found that 50% of spontaneous drawings on digital whiteboards for brainstorming and problem-solving resemble concept maps [62]. Externalizing ideas through concept maps fosters new ideas through associative thinking, and support thinking through cognitive offloading [53]. Numerous studies highlight the positive impact of such representations, with meta-analysis [54] showing that learners benefit significantly more from creating concept maps compared to studying existing ones. Authoring tools can enable learners to view, create, and communicate materials in unique ways conveying contents, structures, and relationships among ideas and concepts.

IV. CODIGEN: USER INTERFACE AND INTERACTION

We built upon the conceptual model (**DP[1-3]**) and designed CODIGEN (**RQ1**), combining a whiteboard and a palette of marking tools, a rich text editor, AI code editor, AI help, AI Chat, and AI code prompter to support authoring, and learning from, educational materials visually and spatially. We ran into many design iterations and gathered informal feedback from three teachers until we reached a functioning tool.

Whiteboard+palettes: Unlike traditional linear interfaces, such as typed documents, whiteboards provide greater flexibility and organization, and have shown to support learning, creativity, problem-solving, and communication [61, 72]. Within CODIGEN, users can create multiple whiteboards, each functioning as a node-based canvas (Figure 1- 1, **DP[1, 3]**). The nodes are small cards, each representing a document with a title and a content body. The body can incorporate multimodal, interactive, and editable content blocks (see Rich Text Editor below). Moreover, users can upload and view media files, primarily images and PDFs. Further, users can arrange cards on the canvas to create meaningful representations. To add a document file (or a note), users can double-click on the canvas; each note will appear as a small card. When a note is created, the note editor will open in an overlay view (Figure 1- 3), and users can have multiple note editors open at once. Users can also open notes in the tabular view (Figure 1- 4). CODIGEN offers a palette of marking tools next to the canvas, enabling users to customize the appearance of their whiteboards (Figure 1- 2). CODIGEN also includes a full-text search tool. Search results relevant to the current whiteboard are highlighted, and users can also select additional note files to add to their boards from the search tool.

Rich text editor: Our editor employs a wysiwyg block-based structure (Figure 1- 4, **DP[1]**), where each block represents a content type. The editor supports a variety of content formats, including headings, paragraphs, images, lists, task items, code snippets, formatting styles, and more. This versatility is particularly advantageous for tasks involving technical writing, concept explanations, and digital content creation. Being interactive and unstructured, CODIGEN rich text editor provides flexibility for both learners and teachers. Learners can engage in writing reflection reports, taking notes, answering exercise questions, and explaining coding processes. Teachers can also adapt materials to better fit their specific needs and contexts, enriching their teaching with explanations, documentation, and examples that support learning.

AI code prompter: The code prompter tool can be accessed through the marking tools and allows users to generate programming materials using default parameters and menus (Figure 2,

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

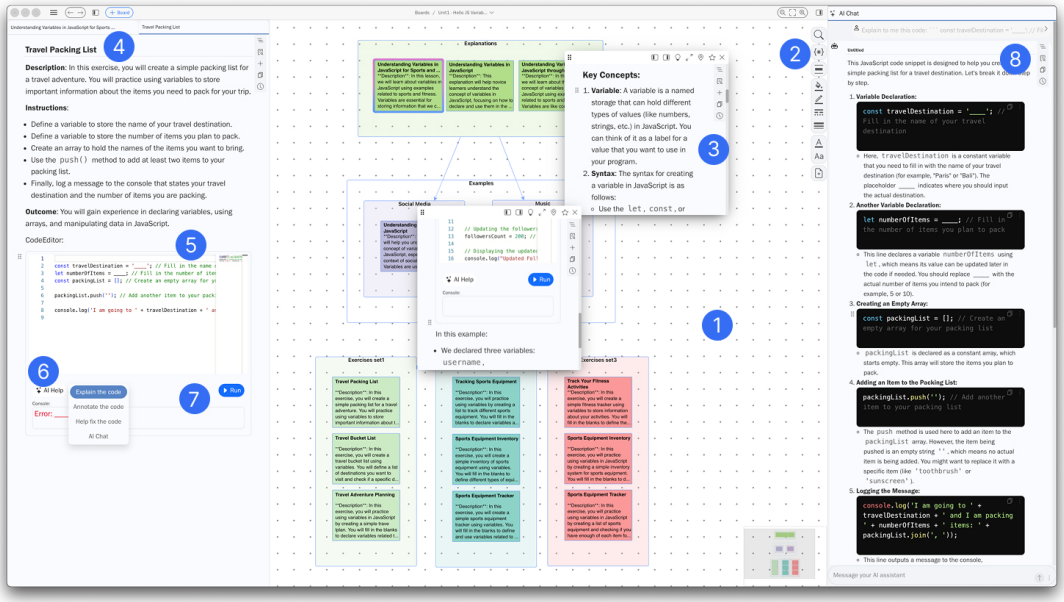


Fig. 1: CODIGEN user interface, with the whiteboard 1 and palettes 2. The whiteboard holds small cards that represent rich-text documents or notes. Users can open and create multiple interactive documents with the rich editor, either through the overlay 3 or the tabular 4 views. The AI Code prompt tool, located in the top-right palette 2, allows users to generate programming materials. The AI Code editor 5 is integrated in the text editor and features built-in AI Help 6 and execution 7. CODIGEN also provides AI Chat 8 which is coordinated with the AI Code editor to display help to users when necessary.

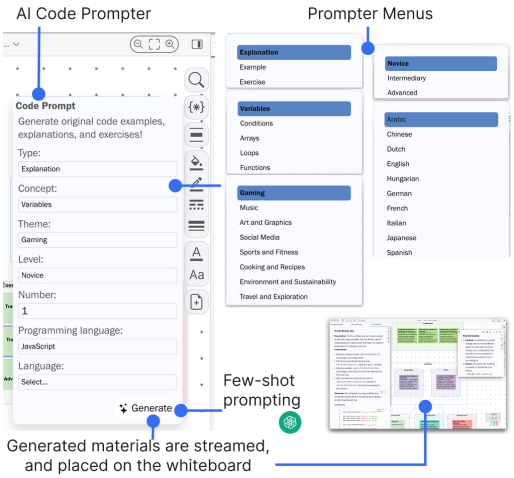


Fig. 2: CODIGEN's AI Prompter view.

DP[2]). The menus include the *type* of material to generate (explanations, examples, and exercises); the programming *concept* of focus (variables, conditions, arrays, loops, and functions); the *theme* of interest (gaming, music, art and graphics, social media, sports and fitness, cooking and recipes, environmental sustainability, and travel and exploration); the learner *level* (novice, intermediate, advanced); the number of items to generate; the programming language (JavaScript); and finally, the selection of *languages* for the generation (Arabic, Chinese, Dutch, etc.). Once the user makes a selection, CODIGEN builds and sends a

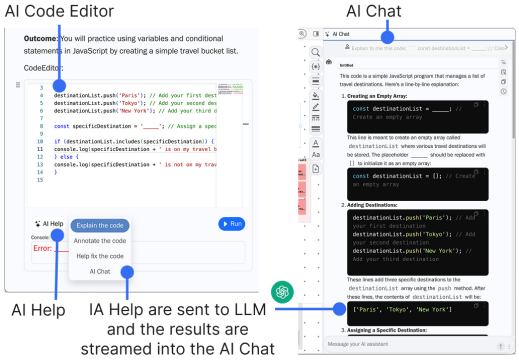


Fig. 3: CODIGEN's AI Code Editor, AI Help, and AI Chat views.

prompt to the LLM. The materials are then generated in realtime as small cards on the whiteboard using a grid layout.

AI code editor: This is an IDE that is embedded in the main text-based editor (Figure 3, **DP[1, 2]**). The code editor is interactive and allows users to write, run, and debug code. It also supports auto-completion and syntax highlighting. Because the code editor is integrated into the main text editor, users can modify the generated materials or add instructions, examples, explanations, and exercises to create well-structured programming materials.

AI helpers: CODIGEN uses GAI prompting techniques to integrate instructive "AI helpers" into the code editor (Figure 3, **DP[2]**). The prompts include "help explain the code," "annotate the code line by line," "help fix the code," and "AI Chat."

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

CODIGEN augments prompts before sending them to the LLM to ensure that AI guides the learners instead of simply providing them with solutions. The AI helpers are integrated with the AI Chat (see AI Chat below). When a user clicks on an AI helper, a prompt is sent directly to the LLM with the editor content, except for "AI Chat," where a prompt is created in the AI Chat tool, allowing the user to modify it before sending it to the LLM. For example, when the user requests help, such as "help explain the code," the AI Chat tool opens, and a prompt is sent to the LLM with the editor content and console. The response is then streamed back to the chat.

AI chat: CODIGEN takes advantage of chat flexibility, allowing users to have interactive conversations with GAI (Figure 3), which can support educational tasks, such as searching information, problem-solving, etc. We added a context window to AI Chat to support conversations, which helps in completing tasks or building solutions step by step. By integrating AI Chat directly into CODIGEN, we ensure in-situ AI assistant. To assist users, CODIGEN incorporates the main text editor into the chat responses generated by the LLM. Users can see code snippets, and have the ability to edit the LLM's output. For instance, if parts of the LLM's response seem irrelevant, users can modify the output as needed. Users can also add titles to the chat responses and drag-and-drop them onto the whiteboard as notes.

A. Implementation

User interface: We designed CODIGEN as a cross-platform desktop app. We used React, TypeScript, Rust, and Tauri. To support user privacy all materials created with CODIGEN are stored on the user's device as markdown files.

LLMs and prompt design: We used the OpenAI web client to prompt GPT-4o-mini and DALL·E 3 [49]. Prompting followed a few-shot with three examples, a template with variables, and a structured output (title, description, instructions, outcome, code blocks). The LLM responded in Markdown. We converted code blocks into our AI Code Editor within the rich text editor.

Code execution: In CODIGEN, we focused on JavaScript due to its popularity and the ability to run it in a local, sandboxed browser environment. Our approach is easily extensible to other concepts (e.g., events, HTML) and programming languages (e.g., Python, Rust) using WebAssembly or server-based interpreters.

V. USER STUDY

We conducted a user study to examine the effectiveness of CODIGEN. To make the study manageable, we focused on two main programming concepts: variables and functions, including declaration and manipulation. We used mixed methods analysis with the following factors and levels: (1) **Learner's Level**: Novice and Intermediate (in JavaScript) and (2) **Survey**: pre-survey (*before*) and post-survey (*after*).

The Level was a between-subject factor. All the participants have a CS background. Novices participants were with little or no formal exposure to JavaScript. Intermediates were participants with some formal exposure to JavaScript, such as having taken an introductory course. The survey (pre- and post-intervention, see appendix A) was a within-subject factor. The pre-survey captured the participants' demographics and backgrounds programming. The post-survey captured the participants' perceived Usability, Utility, Engagement, and Educational Outcome (see subsection V-D). We slightly modified the pre- and post-questionnaires while maintaining the same concepts and level of difficulty. Participants were not aware of the post-questionnaire in advance. We also interviewed 6 participants to gather qualitative feedback.

A. Participants

We recruited 29 participants in total, (i) 23 for the user study [Condition: Novice = 16, Intermediate = 7; Age: 20 - 25 = 19, 25 - 30 = 4; Gender: Female = 8, Male = 15; Ph.D. students = 2, 3rd year university = 6, 5th year university = 15] and (ii) 6 for the interviews 2 teachers [Female: 1, Male: 1], 1 male Ph.D. students, and 3 master students [Female: 2, Male: 1]. All the participants were French native speakers. They voluntarily participated in the study. We did not give a compensation/gift to the participants. All the participants consented to collecting and analyzing their data.

B. Apparatus and Activity Materials

We prepared one whiteboard with 5 cards. One card introduced an explanation of the concept of variables. Another card explained the concept of functions. A third card provides examples of variables. And remaining cards provided two unsolved exercises, which the participants solved (see procedure below).

C. Procedure

The study took place in a classroom and consisted of four main stages, conducted over five sessions (participants' groups). The participants used their laptops.

Opening: We briefed participants on the study, administered a pre-survey for demographics, background, and consent, guided them through installing CODIGEN, and introduced key features (whiteboard, GAI integration, AI Code Prompter, AI Help/Code Editor, and AI Chat). This lasted about 15 minutes.

Programming activity: We asked participants to read two explanation cards (variables, functions) and one example card to understand the concepts and syntax, then solve two exercises. We encouraged them to use AI Help, AI Chat, and the AI Prompter to generate additional explanations, examples, or exercises as needed. We informed them there would be a test afterward, then let them complete the task, which took 30–45 minutes.

Closing: We administered the post-survey to collect participants' perceived Usability, Utility, Engagement, and Educational Outcome, which took about 7 minutes.

Interviews: After the programming sessions, we conducted semi-structured interviews. We asked each participant to use the features of CODIGEN for a few minutes to elicit discussion. Each interview lasted between 30 and 60 minutes.

D. Data Overview and Analysis

We collected 23 pre-survey and 21 post-survey self-reports (two participants missed the post-survey). We measured all

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

responses, except demographics and learning outcomes, on five-point Likert scales.

We applied the SUS questionnaire to assess overall Usability, Ease of Use, and Learnability [10, 31]. We evaluated the Utility of CODIGEN across five sub-factors: Multimodality, GAI Integration, GAI Code Prompter, GAI Help/Code Editor, and GAI Chat (Appendix A).

We used pre–post questionnaires to measure Educational Outcomes. We assessed Confidence through Likert ratings of participants' self-perceived mastery of JavaScript variables and functions. We measured Learning with multiple-choice programming questions scored as true/false, ensuring equivalence between pre- and post-surveys by slightly varying the questions.

We measured Engagement following [40], capturing Positive Feelings (fun, flow), Interest (identification and active engagement), and Future Intents (desire to improve skills and knowledge). These measures indicate participants' engagement with and willingness to adopt CODIGEN.

We audio-recorded the interviews, then transcribed and independently analyzed them.

VI. RESULTS

We report results using both p-values for statistical significance and effect sizes ($small :< .3$, $moderate : .3 - .5$, $large :> .5$) with 95% confidence intervals [39]. We present boxplots showing the median, 1st and 3rd quartiles, whiskers (minimum and maximum), and outliers.

We applied the one-sample Wilcoxon test, a non-parametric method suited to our small sample size and ordinal data, to determine whether the sample median (five-point scale) differs from the neutral value of 3. We calculated the common effect size r by dividing the standardized test statistic by the number of observations [39]. We used the Wilcoxon signed-rank test to compare pre- and post-intervention confidence outcomes. For factorial analyses of count data, we applied a Generalized Linear Mixed-effects Model (GLMM) [9]. We also analyzed Likert ratings with a non-parametric factorial ANOVA using the Aligned Rank Transform (ART) [67].

A. *RQ2: Perceived Usability, Engagement, and Utility*

Perceived Usability. Figure 4 shows the participants scores of the SUS questionnaire by Level, which highlights a positive reaction. We followed [10, 31] and calculated scores (on a scale of 0 to 100) for the SUS [SUS: Q1-10], ease of use [SUS: Q1,2,3,5,6,7,8,9], and learnability [SUS: Q4,10]. The SUS score for all participants was 70.95. The score for the ease of use 71.13. The score for the Learnability was 70.95. The three scores were above the standard SUS average of 68 points. We conducted a one-sample Wilcoxon test to evaluate whether the scores were significantly higher than 68 points. The results showed that users' scores were not significantly higher than 68 points, with a small effect size for total SUS ($p = .48, r = .15, CI = [-.30, .55]$), a small effect size for Ease of Use ($p = .28, r = .24, CI = [-.17, .63]$), and a small effect size for Learnability ($p = .46, r = .16, CI = [-.28, .58]$).

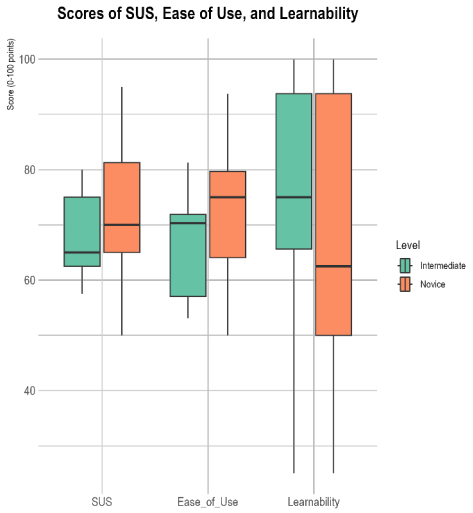


Fig. 4: Boxplot comparison of self-reported Usability by Level.

Engagement: Positive Feelings, Interest, and Future Intents. As described in section V-D, we evaluate three sub-factors of the user Engagement when using educational technology like CODIGEN, namely: Positive Feelings (PF), Interest (IN), and Future Intents (FI). We conducted one-sample Wilcoxon tests to evaluate whether the responses from the participants were significantly higher than the median value of 3 for PF, IN, and FI. The results showed a positive impact of CODIGEN on users across the three sub-factors (Figure 7). The results showed that users' rating were significant with a large effect size for PF Novice ($p < .001, r = .90, CI = [.89, .91]$), PF Intermediate ($p < .001, r = .86, CI = [.74, .95]$), a large effect size for IN Novice ($p < .001, r = .91, CI = [.90, .94]$), IN Intermediate ($p < .001, r = .90, CI = [.90, .94]$), and a large effect size for FI Novice ($p < .001, r = .87, CI = [.73, .96]$) FI Intermediate ($p < .005, r = .92, CI = [.91, .97]$).

Perceived Utility. As reported in section V-D, we designed the post questionnaire to examine the Utility of CODIGEN [Utility: UTQ1-6]. Figure 5, highlights the participant rating of the perceived Utility, showing positive reactions. To further examine the utility of CODIGEN, we designed the questionnaire to capture the Utility of five factors: Multimodality, AI Integration, AI Code Prompter, AI Help/Code Editor, AI Chat.

We conducted a series of one-sample Wilcoxon tests to examine whether the responses from the participants regarding the factors of Utility were significantly higher than the median of 3. Figure 5 summarizes the results. The results showed that the responses from the participants were significant for Multimodality ($p < .001, r = .82, CI = [.73, .90]$), GAI Integration ($p < .001, r = .90, CI = [.90, .92]$), GAI Code Prompter ($p < .001, r = .91, CI = [.90, .96]$), GAI Help/Code Editor ($p < .001, r = .90, CI = [.90, .93]$), GAI Chat ($p < .001, r = .90, CI = [.90, .93]$).

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

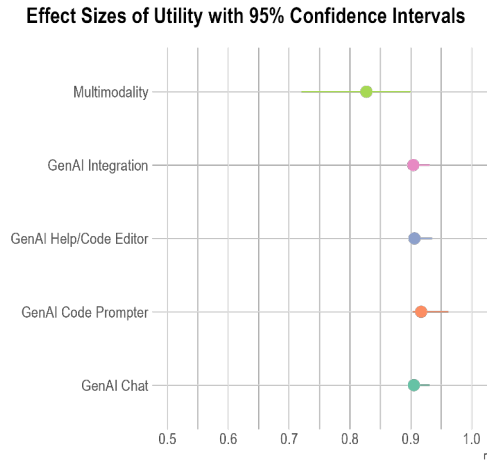


Fig. 5: The r effect sizes of the Utility.

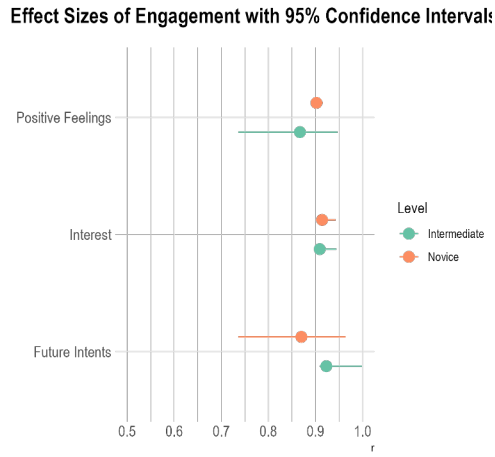


Fig. 7: The r effect sizes of the Engagement.

B. *RQ3: Educational Outcome*

Confidence Outcome. As mentioned in section V-D, we conducted paired comparisons between the pre-post tests to examine the the educational outcomes of users when using CODIGEN. We looked at the users' Confidence Outcome and the Learning Outcome.

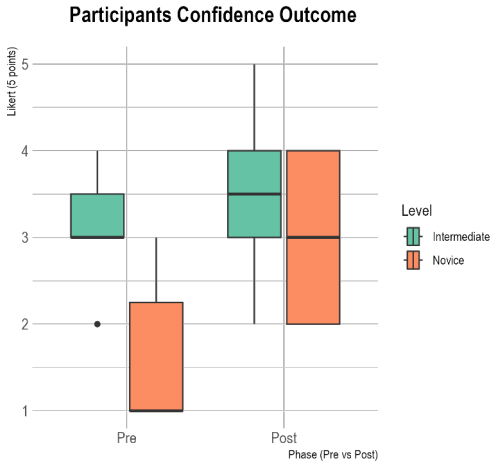


Fig. 6: Boxplots comparison of Confidence Outcome.

The results showed a positive shift in the Confidence Outcome of users when comparing the pre- and post-intervention responses (Figure 6). A wilcoxon signed rank test showed a significant difference between the pre and post responses with large effect size ($p < .001, r = .85, CI = [.68, .87]$). We conducted a $2x2$ factorial analysis of the Likert scores with respect to Level and the Survey (i.e., Intervention: pre versus post), using Aligned Rank Transform (ART) [67]. The results showed that there is a main difference of Likert scores in both Level ($p < .005$) and Intervention ($F_{1,40} = 9.29, p < .001$). We conducted post-hoc pairwise comparisons, using Holm–Bonferroni for adjustments [22]. The results showed a significant difference in the Level for Novice pre- versus post-intervention ($p < .001, estimate = 14.52, SE = 3.49, t.ratio = 4.45$) and no significant difference for Intermediate ($p = n.s$).

Learning Outcomes. Similarly, for the learning outcomes, we conducted a $2x2$ factorial analysis of the count of correct answers with respect to Level and the Intervention (pre/post Survey), using GLMM [9]. We used Level and Survey as fixed effects and Question as a random effect. Figure 8 illustrates the results. The results showed that there is a main difference of the count of correct answers in both Level ($\chi^2(1, N = 20) = 9.11, p < .001$) and Intervention ($\chi^2(1, N = 20) = 8.17, p < .001$). We conducted post-hoc pairwise comparisons, using Holm–Bonferroni for adjustments [22]. The results showed a significant difference for Novice pre- versus post-intervention ($p < .05, estimate = .48, SE = .16, z.ratio = 2.85$) and no significant difference for Intermediate ($p < n.s$).

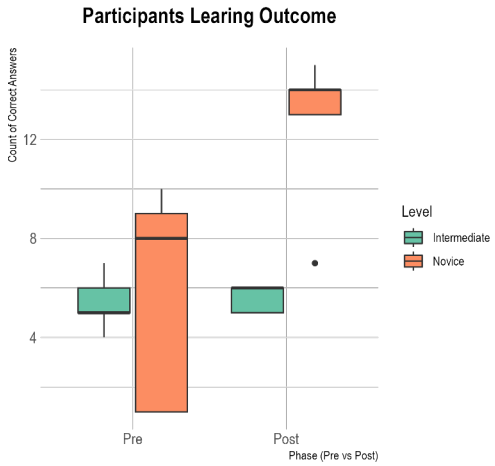


Fig. 8: Boxplots comparison of Learning Outcome.

C. *RQ4: Teachers and Learners Perspectives and Feedback*

Here, we present the qualitative results of the interviews.

Usability. CODIGEN was perceived as interesting and engaging by the participants. They highlighted that the tool offers an innovative

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

approach to learning programming. Unlike traditional methods, CODIGEN allows working on code interactively through multiple exercises, themes, and languages. One participant elaborated: *"I find it very interesting in the sense that it is a tool that is quite playful; one can create a lot of things, one can work on programming not in a classical boring way, but rather in a playful manner, with exercises and different themes. It also covers various [human] languages, which is very interesting —PS1."*

Another participant appreciated the simplicity of CODIGEN and noted it's performance, stating *"It's simple, it's cool and it is fluid -PS2."* even when working on various notes *"which was surprising to me when there were many cards —PS2."*, which enhances the overall user experience,

Utility. Moreover, participants considered CODIGEN as useful and relevant. They particularly appreciated the combination of lessons with an interactive programming environment, which enables them to execute and organize their programming concepts and code. One participant elaborated: *"To put it simply, this is a very interesting tool because it combines two elements. The first element is having lessons with executable code. The second element is being able to organize these codes and different courses. So, for me, it's really interesting —PS3."*

Moreover, the integrated code editor within a rich text editor was valued because it facilitates code comprehension while providing a structured space for organizing educational content. One participant stated *"I find the code editor very useful, especially when it comes to understanding the different parts of the code"*. These features seem to enhance user engagement and make learning more accessible, one learner noted: *"Really, it's a tool to use in the daily life of every student, as it will greatly help them advance and evolve, allowing them to progress quickly and in an organized, well-structured way —PS1."*

GAI Integration. The integration of GAI into the tool was highly valued by the participants. It provides in-situ assistance in programming education, offers smooth interaction, seemingly through coordinated views of the tool.

The integration of GAI within unstructured authoring made the tool flexible to suit the needs of the participants. Unlike other tools that focus solely on code generation or course organization, CODIGEN merges these two aspects, improving the effectiveness of programming learning. One participant stated: *"[...] there are tools that provide GAI aids, and there are tools that organize lessons and structures. So for me, it's the combination of both -the structure and the different steps —that makes this the most interesting aspect here —PS3."*

One teacher elaborated: *"[interesting] especially for explanations and so on, because when we search on Google, we find a lot of information and courses. This is focused. You use just one tool. I think it will be more interesting this way. And you won't have to copy-paste from other courses and so on. I completely imagine that it will enable creating materials that are somewhat personalized and unique compared to others' materials. —PT1"*

One participant specifically highlighted the integration of a system for annotating and explaining code: *"the help [AI Help] with explaining the code, annotating code, and all that seems like a really good idea to me. I think that's really a very good idea —PS2."*. This feature would clarify each line of code and provide detailed guidance for the user.

Suggestions for Improvement. The analysis of user feedback highlighted some improvements. One learner found it challenging to navigate the whiteboard: scroll, pan, zoom, and drag the canvas and its cards, because some interactions require using a hotkey (e.g., *"Ctrl"*) in combination with the mouse or trackpad, the participant said *"Apart from moving around the dashboard, I have to click Ctrl, that's the only thing I found not very easy to use —PS2."* Another learner who is familiar with whiteboard-based tools expressed the needs for tools to support advanced spatial organization of materials on the canvas, such as automated node-link visualization: *"[...] I already use this type of tool [whiteboard] to organize summaries, and I didn't find many features or types of organizational structures —PS3."* One teacher suggested improving the AI Prompter saying: *"in fact, what would be good is that when I ask for two examples, they should be complementary examples —PT2"*. Currently, the tool generates multiple examples, but they might not be entirely complementary.

VII. DISCUSSION

A. Key Findings

Our integration of GAI and authoring techniques for education has shown promising results in enhancing learners' outcomes and engagement. Regarding **RQ1**, we articulated a conceptual framework and built upon it to design CODIGEN, a visuo-spatial authoring tool powered by in-situ GAI assistance to support quality learning in programming education. This work extend previous research into (1) tools powered by LLMs for computer programming education [e.g., 69, 46, 27] and (2) visuospatial authoring interaction with LLMs [e.g., 24, 68, 57]. Concerning **RQ2**, the overall Usability, Ease of Use, and Learnability scores were higher than the standard value of 68, though not significantly. Despite that, we consider the usability of CODIGEN to be quite good, especially considering that the participants did not receive any training on using CODIGEN before the study. The usability was also comparable between the different groups of participants (Novice and Intermediate), with both groups completing the task in about the same amount of time. User Engagement with CODIGEN and its sub-factors, namely Positive Feelings, Interest, and Future Intentions, were significantly strong. The Utility and its sub-factors (Multimodality, GAI Integration, AI Code Prompter, AI Help/Code Editor, and AI Chat) were also significantly strong. We observed this positive reaction from participants during the study and interviews, suggesting that the design choices of CODIGEN were appreciated. Regarding **RQ3**, the results showed a positive shift in the Confidence Outcome pre- versus post-intervention. Similarly, the results indicated a positive shift in the count of correct answers pre- versus post-intervention. Both differences were significant for novice learners but not for intermediates, likely due to the intermediates' prior exposure to the concepts of JavaScript. Finally, the qualitative results from learners and teachers regarding **RQ4** were positive and further support the quantitative findings.

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

B. AI Tools Enhancing Programming Education

Our study has shown that authoring tools powered by AI can significantly improve students' performance in fundamental programming concepts, corroborating previous studies [46, 34, 27]. Interestingly, our approach can support teachers in generating various types of original educational materials, such as explanations, examples, and exercises across diverse themes (e.g., sport, cooking, mystery, art, gaming, etc.). Combining AI tools with unstructured authoring tools enables teachers to tailor content to their unique contexts, break down materials into semantic portions for clarity, ensuring the content is segmented and manageable without overwhelming students. Students can use the same interface to adapt the materials in ways that suit their learning styles, track their progress by marking sections, and add notes or annotations for reflection, collaboration, or discussion with teachers or peers.

C. Student Outcomes and Engagement

Our study demonstrated a positive correlation between AI integration and improved performance in JavaScript tasks. Students reported higher confidence in their programming skills after using CODIGEN. This finding is supported by studies that emphasize the role of technology in skill development [21]. Additionally, findings on the impact of AI on programming further corroborate these results [e.g., 46, 69, 27, 34]. The multimodal capabilities of our tool—integrating rich text editors, code, whiteboard, IA Chat, and more—were particularly appreciated for enhancing the programming experience of students. Student engagement metrics, measured by positive feelings and future intentions, indicated a high level of satisfaction. Students found CODIGEN useful and reported increased interest in the subject matter when comparing pre- to post-intervention data. This aligns with research showing that interactive learning methods boost engagement and motivation [e.g., 41, 40].

D. Challenges and Limitations

Despite these positive outcomes, challenges remain. Issues, such as over-reliance on AI were noted by some students. For example, one student elaborated, stating that they did nothing because the IA helped considerably with performing the task. While we added guardrails to prevent the AI from revealing solutions outright, we retained the "IA chat" feature, which allows users to refine and tweak the prompt before sending it to the LLM. Advanced users can still prompt the LLM for a solution. These limitations highlight the need for balanced use of AI tools to prevent dependency and ensure effective outcomes [e.g., 2, 26].

Moreover, throughout the process of exercises' generation, several limitations of the LLMs were identified. One notable issue was its inability to adhere to the specified instructions within the prompt. For example, despite a clear directive to generate exercises using the fill-in-the-blanks strategy while providing examples (i.e., few shot prompting [38, 65]), at times the model's output failed to include any blank sections. This mismatch suggests a gap between the task and the model's ability to execute it properly, especially after multiple generations. Another limitation emerged in cases where the model was expected to generate executable code blocks with blank sections. Instead of producing the required outcome, the output frequently contained only comments. These comments, while syntactically correct, failed to fulfill the core objective of generating code blocks with blank sections. This behavior might be linked to the hyper-parameters of the model, such as temperature, or the hallucination [6], limiting its ability to consistently follow the prompt.

The environmental impact of large language models (LLMs) involves high energy consumption and carbon footprint, during training and inference. Our tool enables the creation of educational materials that can be shared and reused between teachers and learners, reducing the need for repeated model usage and thereby lowering environmental costs. Additionally, fine-tuning smaller or domain-specific language models [33] can reduce computational demands and energy consumption while maintaining effectiveness. These strategies can offer a sustainable approach to integrating AI in education.

E. Future Directions

Our findings can be attributed to both the tool's features and the materials (content). Performing some A/B testing (e.g., control groups) can help trace the impact. However, we emphasize that both the materials and the tool are intimately related. Also, future research can explore personalized AI tools that adapt to individual learning paces and styles. Technological improvements, such as enhancing prompt engineering features, node-based visualization, and user-friendliness of AI interfaces are crucial for maximizing educational benefits. Finally, inconsistencies in the LLM outputs highlight the need for further refinement of the models (e.g., fine-tuning) and the prompting strategies to ensure that the results align more closely with the expected outcomes.

VIII. CONCLUSION

We articulated a conceptual framework to support the design and use of visual authoring tools powered by LLMs. We introduced CODIGEN, a visual authoring tool powered by LLMs to support programming education. CODIGEN offers features to integrate with LLMs, process natural language, and enhance programming education using different learning strategies, including explanations, examples, variations of small fill-in-the-blank exercises, solutions, feedback, and personalization.

We designed and conducted a user study of CODIGEN with 29 participants. Our findings showed that integrating LLMs into authoring tools can effectively provide in-situ assistance for programming education. Most importantly, CODIGEN led to a significant improvement in participant educational outcomes when comparing pre- and post-intervention results, in particular for novice learners. We hope our work provides new insights for future computer programming education research and adds value to the development of LLM-powered authoring tools.

ACKNOWLEDGMENTS

We thank all the teachers and students who participated in the study, as well as the anonymous reviewers for their valuable comments and suggestions.

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] H. S. Alenezi and M. H. Faisal. "Utilizing Crowdsourcing and Machine Learning in Education: Literature Review". In: *Education and Information Technologies* 25.4 (July 2020), pp. 2971–2986.

[2] M. Amoozadeh, D. Nam, D. Prol, A. Alfageeh, J. Prather, M. Hilton, S. Srinivasa Ragavan, and A. Alipour. "Student-AI Interaction: A Case Study of CS1 Students". In: *Proceedings of the 24th Koli Calling International Conference on Computing Education Research*. Nov. 2024, pp. 1–13.

[3] I. Arawjo, P. Vaithilingam, M. Wattenberg, and E. Glassman. "ChainForge: An Open-Source Visual Programming Environment for Prompt Engineering". In: *Adjunct Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Oct. 2023, pp. 1–3.

[4] S. O. Bada and S. Olusegun. "Constructivism learning theory: A paradigm for teaching and learning". In: *Journal of Research & Method in Education* 5.6 (2015), pp. 66–70.

[5] R. S. Baker, A. T. Corbett, K. R. Koedinger, and A. Z. Wagner. "Off-Task Behavior in the Cognitive Tutor Classroom: When Students "Game the System"". In: *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*. 2004, pp. 383–390.

[6] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Wilie, H. Lovenia, Z. Ji, T. Yu, W. Chung, Q. V. Do, Y. Xu, and P. Fung. *A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity*. Nov. 2023. arXiv: 2302.04023 [cs].

[7] D. Barr, J. Harrison, and L. Conery. "Computational Thinking: A Digital Age Skill for Everyone." In: *Learning & Leading with Technology* 38.6 (2011), pp. 20–23.

[8] M. L. Bernacki, M. J. Greene, and N. G. Lobczowski. "A Systematic Review of Research on Personalized Learning: Personalized by Whom, to What, How, and for What Purpose(s)?" In: *Educational Psychology Review* 33.4 (Dec. 2021), pp. 1675–1715.

[9] B. M. Bolker, M. E. Brooks, C. J. Clark, S. W. Geange, J. R. Poulsen, M. H. H. Stevens, and J.-S. S. White. "Generalized linear mixed models: a practical guide for ecology and evolution". In: *Trends in ecology & evolution* 24.3 (2009), pp. 127–135.

[10] J. Brooke et al. "SUS-A Quick and Dirty Usability Scale". In: *Usability evaluation in industry* 189.194 (1996), pp. 4–7.

[11] C. S. Cheah. "Factors Contributing to the Difficulties in Teaching and Learning of Computer Programming: A Literature Review". In: *Contemporary Educational Technology* 12.2 (May 2020), ep272.

[12] T. K. F. Chiu. "The Impact of Generative AI (GenAI) on Practices, Policies and Research Direction in Education: A Case of ChatGPT and Midjourney". In: *Interactive Learning Environments* 0.0 (2023), pp. 1–17.

[13] B. Curtis. "Fifteen years of psychology in software engineering: Individual differences and cognitive science". In: *Proceedings of the 7th international conference on Software engineering*. 1984, pp. 97–106.

[14] A. Cypher, M. Dontcheva, T. Lau, and J. Nichols. *No code required: giving users tools to transform the web*. 2010.

[15] P. Denny, J. Prather, B. A. Becker, J. Finnie-Ansley, A. Hellas, J. Leinonen, A. Luxton-Reilly, B. N. Reeves, E. A. Santos, and S. Sarsa. "Computing Education in the Era of Generative AI". In: *Communications of the ACM* 67.2 (Feb. 2024), pp. 56–67.

[16] G. Di Fede, D. Rocchesso, S. P. Dow, and S. Andolina. "The Idea Machine: LLM-based Expansion, Rewriting, Combination, and Suggestion of Ideas". In: *Proceedings of the 14th Conference on Creativity and Cognition*. June 2022, pp. 623–627.

[17] L. Fiorella and R. E. Mayer. "Eight Ways to Promote Generative Learning". In: *Educational Psychology Review* 28.4 (Dec. 2016), pp. 717–741.

[18] G. E. Gonzalez, D. A. S. Moran, S. Houde, J. He, S. I. Ross, M. J. Muller, S. Kunde, and J. D. Weisz. "Collaborative Canvas: A Tool for Exploring LLM Use in Group Ideation Tasks." In: *IUI Workshops*. 2024.

[19] R. S. Grabinger and J. C. Dunlap. "Rich Environments for Active Learning: A Definition". In: *Research in Learning Technology* 3.2 (1995).

[20] D. Guo, S. Zhang, K. L. Wright, and E. M. McTigue. "Do You Get the Picture? A Meta-Analysis of the Effect of Graphics on Reading Comprehension". In: *AERA Open* 6.1 (Jan. 2020), p. 2332858420901696.

[21] A. Haleem, M. Javaid, M. A. Qadri, and R. Suman. "Understanding the Role of Digital Technologies in Education: A Review". In: *Sustainable Operations and Computers* 3 (Jan. 2022), pp. 275–285.

[22] S. Holm. "A simple sequentially rejective multiple test procedure". In: *Scandinavian journal of statistics* (1979), pp. 65–70.

[23] E. Jiang, K. Olson, E. Toh, A. Molina, A. Donsbach, M. Terry, and C. J. Cai. "PromptMaker: Prompt-based Prototyping with Large Language Models". In: *CHI Conference on Human Factors in Computing Systems Extended Abstracts*. Apr. 2022, pp. 1–8.

[24] P. Jiang, J. Rayan, S. P. Dow, and H. Xia. "Graphologue: Exploring Large Language Model Responses with Interactive Diagrams". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Oct. 2023, pp. 1–20.

[25] S.-H. Jin. "Visual Design Guidelines for Improving Learning from Dynamic and Interactive Digital Text". In: *Computers & Education* 63 (Apr. 2013), pp. 248–258.

[26] M. Kazemitabaar, X. Hou, A. Henley, B. J. Ericson, D. Weintrop, and T. Grossman. "How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment". In: *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research*. Nov. 2023, pp. 1–12.

[27] M. Kazemitabaar, R. Ye, X. Wang, A. Z. Henley, P. Denny, M. Craig, and T. Grossman. *CodeAid: Evaluating a Classroom Deployment of an LLM-based Programming Assistant That Balances Student and Educator Needs*. Jan. 2024. arXiv: 2401.11314 [cs].

[28] H. Keuning, J. Jeuring, and B. Heeren. "A Systematic Literature Review of Automated Feedback Generation for Programming Exercises". In: *ACM Transactions on Computing Education* 19.1 (Mar. 2019), pp. 1–43.

[29] J. R. Kirby and M. J. Lawson. *Enhancing the Quality of Learning: Dispositions, Instruction, and Learning Processes*. May 2012.

[30] D. R. Krathwohl. "A Revision of Bloom's Taxonomy: An Overview". In: *Theory Into Practice* 41.4 (Nov. 2002), pp. 212–218.

[31] J. R. Lewis and J. Sauro. "The Factor Structure of the System Usability Scale". In: *Human Centered Design*. Vol. 5619. 2009, pp. 94–103.

[32] H. Li, J. Zhang, C. Li, and H. Chen. "Resdsql: Decoupling Schema Linking and Skeleton Parsing for Text-to-Sql". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 2023, pp. 13067–13075.

[33] Y. Li, S. Bubeck, R. Eldan, A. Del Giorno, S. Gunasekar, and Y. T. Lee. *Textbooks Are All You Need II: Phi-1.5 Technical Report*. Sept. 2023. arXiv: 2309.05463 [cs].

[34] M. Liffiton, B. Sheese, J. Savelka, and P. Denny. *CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes*. Aug. 2023. arXiv: 2308.06921 [cs].

[35] W. M. Lim, A. Gunasekara, J. L. Pallant, J. I. Pallant, and E. Pechenkina. "Generative AI and the Future of Education: Ragnarök or Reformation? A Paradoxical Perspective from Management Educators". In: *The International Journal of Management Education* 21.2 (July 2023), p. 100790.

[36] "Limitations of Student Control: Do Students Know When They Need Help?" In: *Lecture Notes in Computer Science*. 2000, pp. 292–303.

[37] A. Lishinski, A. Yadav, and R. Enbody. "Students' Emotional Reactions to Programming Projects in Introduction to Programming: Measurement Approach and Influence on Learning Outcomes". In: *Proceedings of the 2017 ACM Conference on International Computing Education Research*. Aug. 2017.

[38] V. Liu and L. B. Chilton. "Design Guidelines for Prompt Engineering Text-to-Image Generative Models". In: *CHI Conference on Human Factors in Computing Systems*. Apr. 2022, pp. 1–23.

[39] S. S. Mangiafico. *R Handbook: Summary and Analysis of Extension Program Evaluation in R*. https://rcompanion.org/handbook/.

[40] K. Marquardt and L. Happe. "Saving Bees with Computer Science: A Way to Spark Enthusiasm and Interest through Interdisciplinary Online Courses". In: *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. June 2023, pp. 145–151.

[41] R. E. Mayer. *The Cambridge Handbook of Multimedia Learning*. July 2014.

[42] R. P. Medeiros, G. L. Ramalho, and T. P. Falcão. "A Systematic Literature Review on Teaching and Learning Introductory Programming in Higher Education". In: *IEEE Transactions on Education* 62.2 (2018), pp. 77–90.

[43] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi. "Automated Grading and Feedback Tools for Programming Education: A Systematic Review". In: *ACM Transactions on Computing Education* 24.1 (Mar. 2024), pp. 1–43.

[44] Microsoft. *GitHub Copilot · Your AI Pair Programmer*. https://github.com/features/copilot. 2025.

[45] D. M. Mohaghegh and M. McCauley. "Computational Thinking: The Skill Set of the 21st Century". In: (2016).

[46] D. Nam, A. Macvean, V. Hellendoorn, B. Vasilescu, and B. Myers. "Using an LLM to Help With Code Understanding". In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. Apr. 2024, pp. 1–13.

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

[47] E. Nijkamp, B. Pang, H. Hayashi, L. Tu, H. Wang, Y. Zhou, S. Savarese, and C. Xiong. *CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis*. Feb. 2023. arXiv: 2203.13474 [cs].
[48] OpenAI. *ChatGPT*. https://chatgpt.com. 2024.
[49] OpenAI. *Openai/Openai-Node: The Official Node.Js / Typescript Library for the OpenAI API*. https://github.com/openai/openai-node.
[50] J. C. Paiva, J. P. Leal, and Á. Figueira. "Automated Assessment in Computer Science Education: A State-of-the-Art Review". In: *ACM Transactions on Computing Education* 22.3 (Sept. 2022), pp. 1–40.
[51] P. Pirolli and M. Recker. "Learning Strategies and Transfer in the Domain of Programming". In: *Cognition and Instruction* 12.3 (1994), pp. 235–275. JSTOR: 3233761.
[52] N. Pirttinen, V. Kangas, I. Nikkarinen, H. Nygren, J. Leinonen, and A. Hellas. "Crowdsourcing Programming Assignments with CrowdSorcerer". In: *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education*. July 2018, pp. 326–331.
[53] E. F. Risko and S. J. Gilbert. "Cognitive Offloading". In: *Trends in Cognitive Sciences* 20.9 (Sept. 2016), pp. 676–688.
[54] N. L. Schroeder, J. C. Nesbit, C. J. Anguiano, and O. O. Adesope. "Studying and Constructing Concept Maps: A Meta-Analysis". In: *Educational Psychology Review* 30.2 (June 2018), pp. 431–455.
[55] E. Stern, C. Aprea, and H. G. Ebner. "Improving Cross-Content Transfer in Text Processing by Means of Active Graphical Representation". In: *Learning and Instruction* 13.2 (Apr. 2003), pp. 191–203.
[56] "Student Strategies for Learning Programming from a Computational Environment". In: *Lecture Notes in Computer Science*. 1992, pp. 382–394.
[57] S. Suh, B. Min, S. Palani, and H. Xia. "Sensecape: Enabling Multilevel Exploration and Sensemaking with Large Language Models". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. Oct. 2023, pp. 1–18.
[58] J. J. G. Van Merriënboer and P. Ayres. "Research on Cognitive Load Theory and Its Design Implications for E-Learning". In: *Educational Technology Research and Development* 53.3 (Sept. 2005), pp. 5–13.
[59] T. van. "The Signaling (or Cueing) Principle in Multimedia Learning". In: *The Cambridge Handbook of Multimedia Learning*. 2nd ed. 2014, pp. 263–278.
[60] J. Wagemans, J. H. Elder, M. Kubovy, S. E. Palmer, M. A. Peterson, M. Singh, and R. von der Heydt. "A Century of Gestalt Psychology in Visual Perception I. Perceptual Grouping and Figure-Ground Organization". In: *Psychological bulletin* 138.6 (Nov. 2012), pp. 1172–1217.
[61] J. Walny, S. Carpendale, N. Henry Riche, G. Venolia, and P. Fawcett. "Visual Thinking in Action: Visualizations as Used on Whiteboards". In: *IEEE Transactions on Visualization and Computer Graphics* 17.12 (Dec. 2011), pp. 2508–2517.
[62] J. Walny, S. Carpendale, N. Henry Riche, G. Venolia, and P. Fawcett. "Visual Thinking in Action: Visualizations as Used on Whiteboards". In: *IEEE Transactions on Visualization and Computer Graphics* 17.12 (Dec. 2011), pp. 2508–2517.
[63] M. Wang, B. Wu, P. A. Kirschner, and J. Michael Spector. "Using Cognitive Mapping to Foster Deeper Learning with Complex Problems in a Computer-Based Environment". In: *Computers in Human Behavior* 87 (Oct. 2018), pp. 450–458.
[64] Weforum. *The Future of Jobs Report 2023*. https://www.weforum.org/publications/the-future-of-jobs-report-2023/.
[65] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. Feb. 2023. arXiv: 2302.11382 [cs].
[66] B. Wisniewski, K. Zierer, and J. Hattie. "The power of feedback revisited: A meta-analysis of educational feedback research". In: *Frontiers in psychology* 10 (2020), p. 487662.
[67] J. O. Wobbrock, L. Findlater, D. Gergle, and J. J. Higgins. "The aligned rank transform for nonparametric factorial analyses using only anova procedures". In: *Proceedings of the SIGCHI conference on human factors in computing systems*. 2011, pp. 143–146.
[68] X. Xu, J. Yin, C. Gu, J. Mar, S. Zhang, J. L. E, and S. P. Dow. "Jamplate: Exploring LLM-Enhanced Templates for Idea Reflection". In: *Proceedings of the 29th International Conference on Intelligent User Interfaces*. Mar. 2024, pp. 907–921.
[69] L. Yan, A. Hwang, Z. Wu, and A. Head. "Ivie: Lightweight Anchored Explanations of Just-Generated Code". In: *Proceedings of the CHI Conference on Human Factors in Computing Systems*. May 2024, pp. 1–15.
[70] A. Yuan, A. Coenen, E. Reif, and D. Ippolito. "Wordcraft: Story Writing With Large Language Models". In: *27th International Conference on Intelligent User Interfaces*. Mar. 2022, pp. 841–852.
[71] M. Ez-Zaouia and R. Carrillo. "DotWrangler: A Method for Assessing Fluency, Originality, and Flexibility of Concept Maps and Diagrams at Scale". In: *Proceedings of the 15th International Conference on Computer Supported Education*. Vol. 2. 2023, pp. 442–450.
[72] M. Ez-zaouia and R. Carrillo. "The Group Folding Effect: The Role of Collaborative Process Structuring and Social Interaction in Group Work". In: *ACM Transactions on Computer-Human Interaction* (Sept. 2023), p. 3622783.
[73] M. Ez-Zaouia, I. Marfisi-Schottman, and C. Mercier. "Authoring tools: the road to democratizing augmented reality for education". In: *Proceedings of the 15th International Conference on Computer Supported Education*. Vol. 1. 2023, pp. 115–127.
[74] L. Zhang, J. D. Basham, and S. Yang. "Understanding the Implementation of Personalized Learning: A Research Synthesis". In: *Educational Research Review* 31 (Nov. 2020), p. 100339.

APPENDIX

A. Questionnaires

1) Pre

Educational Outcome

LOQ1 = "How confident are you in your skills with variables and functions in JavaScript? [Choose:]"

LOPRQ2 = "What is the correct way to declare a variable that contains a number in JavaScript?"

LOPRQ3 = "What will the following code display?
a = 5;
b = 10;
c = a + b;
console.log(c);"

LOPRQ4 = "What is the difference between the keywords const and let?"

LOPRQ5 = "What is the correct declaration of a function multiply that multiplies two numbers in JavaScript?"

LOPRQ6 = "What is the correct way to call a function multiply that multiplies two numbers in JavaScript?"

2) Post:

Perceived Utility

UTQ1 = "I like the integration with generative AI: [Choose:]"

UTQ2 = "I like the multimodal aspect of content creation (images, visualization, code, text, etc.) [Choose:]"

UTQ3 = "I like the way of applying styles, such as map colors, themes, etc. [Choose:]"

UTQ4 = "I think the integration of the generator for explanations/exercises/examples is useful [Choose:]"

UTQ5 = "I think the integration of generative AI assistance in the code editor is useful [Choose:]"

UTQ6 = "I think the integration of chat with generative AI is useful [Choose:]"

Educational Outcome

LOQ1 = "How confident are you in your skills with variables and functions in JavaScript? [Choose:]"

LOPSQ2 = "What is the correct way to declare a variable that contains a string in JavaScript?"

LOPSQ3 = "What will the following code display?
a = 5;
b = a;

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.

c = 10;
d = a + b + c;
console.log(d);”

LOPSQ4 = ”What is the difference between the keywords const and let?”

LOPSQ5 = ”What is the correct declaration of an add function that adds three numbers in JavaScript?”

LOPSQ6 = ”What is the correct way to call an add function that adds three numbers in JavaScript?”

Engagement (Positive Feelings, Interest, Future Intents)

PFQ1 = ”It was nice to explore the content this way [Choose:]”

PFQ2 = ”This way of showing things really impressed me! [Choose:]”

PFQ3 = ”I feel like time flies when exploring content this way! [Choose:]”

PFQ4 = ”It would be great if we used tools like this more often! [Choose:]”

PFQ5 = ”I loved discovering this kind of content presentation! [Choose:]”

INQ1 = ”I would recommend this kind of content presentation to others! [Choose:]”

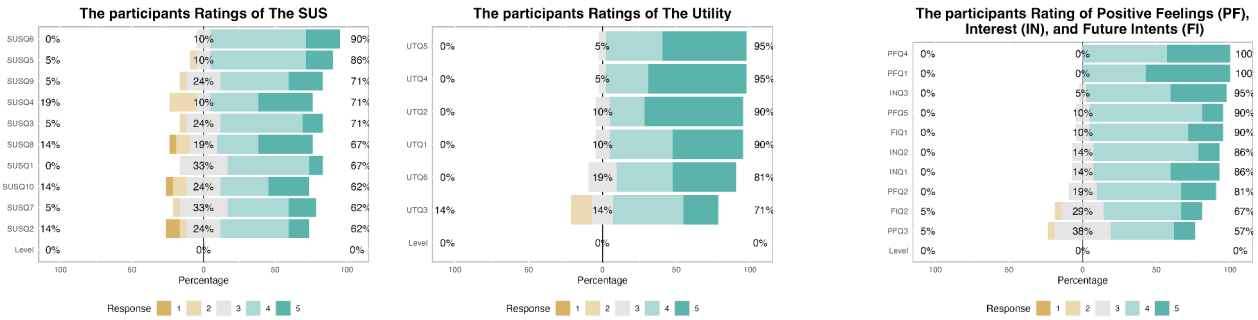
INQ2 = ”This way of presenting the content sparked my interest in the subject [Choose:]”

INQ3 = ”I feel like I learned things by exploring the content this way! [Choose:]”

FIQ1 = ”I will tell others about this way of presenting content [Choose:]”

FIQ2 = ”I can be creative by exploring content this way! [Choose:]”

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.



(a) SUS (b) Utility (c) Engagement

Fig. 9: The participants rating of the SUS, Utility, and Engagement questionnaires.

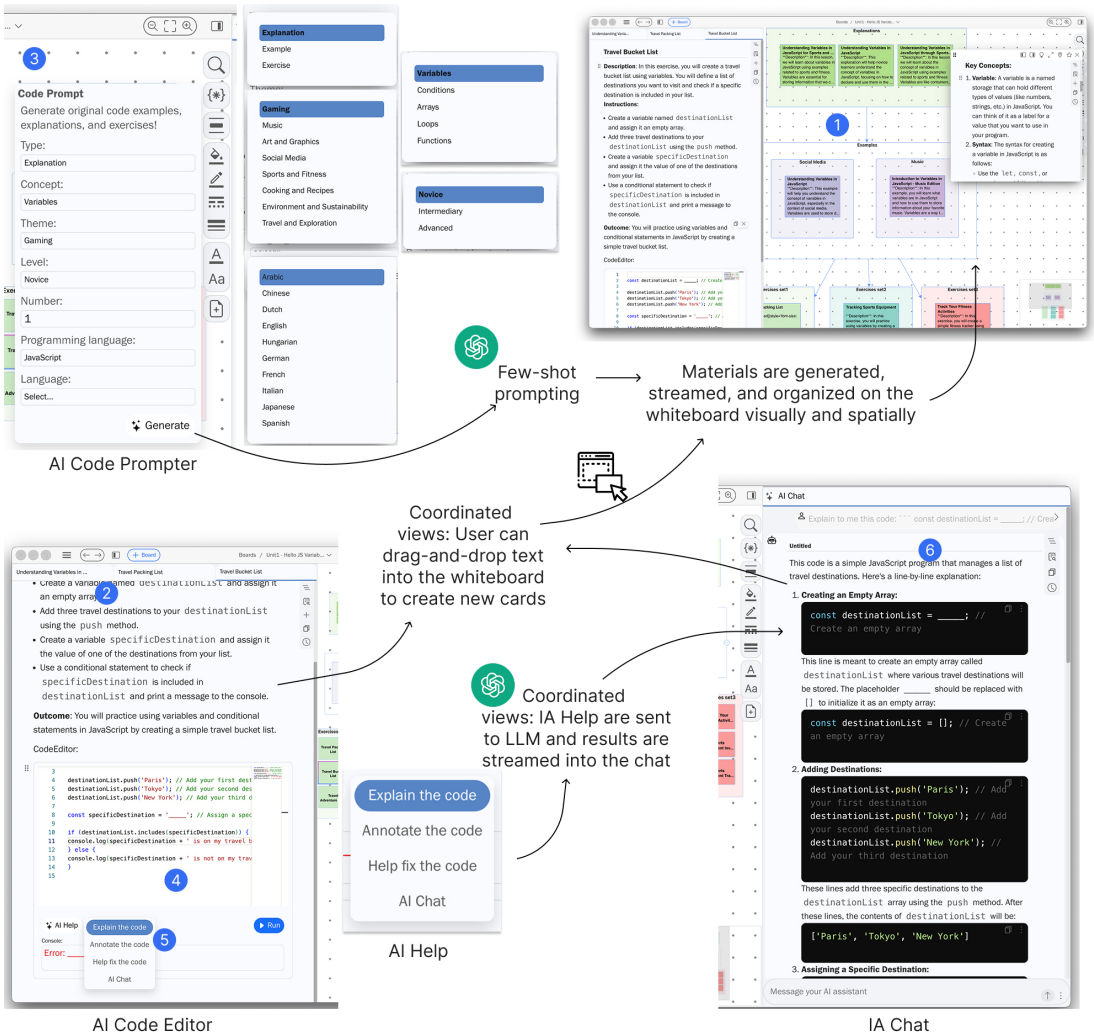


Fig. 10: Coordinated views of CODIGEN

Authorized licensed use limited to: Universidade Federal de Sergipe. Downloaded on September 15,2026 at 02:58:50 UTC from IEEE Xplore. Restrictions apply.